



Group: Lois FC

Player Similarity System

Ewurakua Amoah 74492028

Gabriel Akurang 35882028

Haris Tiyumtaba Issah 41942028

CSIS Department, Ashesi University

MATH 212: Linear Algebra

Dr Ayawoa Sitsope Dagbovie

August 11th, 2026

Introduction

In football, player substitutions and transfers can make or break a team's trajectory. For example, in the 2014 World Cup final at the 88th minute, a player called Gotze was subbed in to replace the World Cup's top goalscorer on the field, Miroslav Klose. This was a surprise to many, as he had not been a key player on the team; however, Gotze went on to score the World Cup-winning goal in the 113th minute for Germany. On the other hand, many games have been lost, and many winning streaks have been broken due to poor substitutions or transfers.

Before, players were chosen based on intuition and by manually checking files. This was long and tedious and could cause many issues, especially on the ground, due to inconsistencies in data and estimates. This has caused many controversies and backlash in the football space. These days, algorithms are being built to automate this process, allowing coaches to find players from other teams who play similarly to their current players or to quickly make substitutions that fit the game plan.

The problem being investigated is the accurate identification of the most similar football players based on metrics such as speed and agility. This is relevant especially to coaches and other sports team officials. Our group seeks to discover the most accurate way to calculate player similarity and enable team officials to quickly find the most similar players and compare their individual metrics.

In this report, we shall break down the theories we used, such as Euclidean distance and radar charts, and then provide some history of Euclidean distance and how its use has changed over time. We shall also justify its use over other similarity approaches, such as the Manhattan and Cosine algorithms. Then we shall break down our methods and thinking processes, highlighting the linear algebra concepts used. This shall be supported by well-explained code snippets. Our results will be highlighted, and we shall conclude by sharing key insights from this project. Further developments which we did not pursue would also be given and briefly discussed.

Background

Key Concepts

Euclidean Distance:

It is defined as “the shortest path between two points in Euclidean space.”(Chugani, 2024)

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Where p and q are two vectors. It is an equation originally derived from the Pythagoras theorem by the Greek Mathematician Euclid in 300BC, which has become one of the foundations of geometry, data science, and many other fields of study. The process will be explained more in the methods section.

This algorithm is now used for K-means clustering, K-nearest neighbours, and many other machine learning techniques. It is typically preferred for its ease of use and ability to rotate directions whilst producing the same answers.

Normalization:

This is defined as “the process of adjusting values measured on different scales to a common scale”(Shaibu, 2024). We chose to use it because the Euclidean distance is very sensitive to magnitudes, which was a big issue in the project because certain metrics might be scored out of a higher amount than others which could skew the results. As a result we decided to normalize the data using mean normalization (Z-score normalization). The reason we chose mean normalization was because, we are comparing players amongst themselves, as compared to min max scaling, which is used to normalize entities against a particular benchmark.

Radar Map:

This is a visualization technique used to compare multiple features that start from a common point. It is mainly used “for performance evaluations where multiple metrics need to be assessed simultaneously.”(Highsoft, n.d) For example, employee performance reviews.

Weights:

Naturally, the Euclidean distance does not hold different standards to different features. However, that matters in football and a player with high speed and low accuracy cannot be the same as a player with low speed but high accuracy. To combat this, we decided to utilize weights to ensure that players who play similar positions are considered nearer to each other.

The best option settled on was to multiply a scalar by a set of penalties which were specific to the position a player played at. For example, so that a goal keeper would not be considered the next best option for a striker.

Methods

To solve the problem, the players' data is read from a csv file named "synthetic_players.csv" and loaded into a data frame. Their raw performance attributes are transformed into standardized row vectors with each column assigned to their respective stats. Attached in the Appendix, includes Figure M1 of the code snippet used to create the row vectors.

This is the matrix for all the players, whereby each row represents a player and the columns are the scalars for the various features.

```
[ [67.6  70.2  86.1  45.4  50.8  40.4  60.6  59.1  0.22  0.26]
  [69.8  88.5  81.2  51.  50.9  40.2  75.6  82.3  0.  0.08]
  [59.4  57.  75.2  46.3  55.5  43.6  77.6  85.6  0.05  0.09]
  [60.7  57.2  71.  41.  56.1  49.7  75.1  87.2  0.19  0.16]
  [58.8  77.7  87.9  37.4  60.1  28.6  76.6  72.5  0.2  0.12]
  [68.2  73.7  68.  58.7  35.9  38.3  68.6  76.  0.42  0.33]
  [83.4  62.9  65.1  45.2  65.7  33.8  67.8  78.8  0.  0.1 ]
  [51.6  59.4  70.1  50.8  51.5  39.7  68.  83.8  0.04  0.12]
  [70.7  76.2  56.8  39.7  85.5  47.1  60.5  82.8  0.  0.  ] ]
```

The player positions are also encoded into various integer values using a custom position dictionary for ease of identification when applying the weights to the final euclidean distance value.

```
encoding = {"GK": 0, "CM": 1, "CAM": 2, "FB": 3, "CB": 4, "ST": 5}
```

A 6x6 matrix (P) representing positional differences was utilized to account for differences in positions. Identical positions received a 0 value, while dissimilar positions received a higher penalty. The higher the value, the more different their positions are from one another.

```
position_distance = np.array(
    [
        # GK CM CAM FB CB ST
        [0, 4, 4, 4, 3, 4], # GK
        [4, 0, 1, 2, 3, 2], # CM
        [4, 1, 0, 3, 4, 1], # CAM
        [4, 2, 3, 0, 1, 4], # FB
        [3, 3, 4, 1, 0, 4], # CB
        [4, 2, 1, 4, 4, 0], # ST
    ]
)
overall_weight = 3 # directly multiply this to the matrix
position_distance = position_distance * overall_weight
```

Based on the relevance attached to the positions of the players, a weight known as the overall weight is multiplied onto the position distance values. The smaller the overall_weight, the lesser the effect of the position on the final Euclidean distance and vice versa. This scalar weight is multiplied through the matrix P to adjust the value of the positional weights appropriately.

[0	12	12	12	9	12]
[12	0	3	6	9	6]
[12	3	0	9	12	3]
[12	6	9	0	3	12]
[9	9	12	3	0	12]
[12	6	3	12	12	0]

In the example above, a scalar multiplication was done on the position distance matrix by an overall_weight of 3.

Each player attribute has its own range of values. Thus, attributes that naturally have high values such as pace and dribbling ranging from 50 to 90, will dominate the final euclidean value. This will overshadow the lesser stats like “goals_per90” ranging from 0 to 1. To get rid of this bias, we make use of a transformation that standardizes each feature dimension by subtracting the mean and dividing by the standard deviation for each column.

$x_{scaled} = \frac{x - \mu}{\sigma}$. Attached in the Appendix, includes Figure M2 of the code snippet used to scale down the values.

This is how the matrices are presented after it has been scaled.

```
[ [ 0.35  0.62  1.56  0.03  0.19 -1.14 -1.41 -0.44  0.03  1.28]
  [ 0.46  1.6   1.19  0.25  0.19 -1.16  0.26  0.76 -0.89 -0.37]
  [-0.07 -0.08  0.73  0.07  0.37 -0.95  0.48  0.93 -0.68 -0.28]
  [ 0.    -0.07  0.41 -0.13  0.4  -0.59  0.2   1.01 -0.1   0.37]
  [-0.1   1.02  1.7   -0.27  0.56 -1.85  0.37  0.25 -0.06 -0.   ]
  [ 0.38  0.81  0.18  0.54 -0.4  -1.27 -0.52  0.43  0.86  1.93]
  [ 1.15  0.24 -0.05  0.03  0.78 -1.54 -0.61  0.58 -0.89 -0.19]
  [-0.46  0.05  0.34  0.24  0.21 -1.19 -0.59  0.84 -0.72 -0.   ]
  [ 0.51  0.94 -0.68 -0.18  1.56 -0.74 -1.42  0.78 -0.89 -1.1 ]
```

With the features now scaled and bias reduced, the euclidean distance is now calculated. For any two given players, the difference for each feature is squared and summed across the various attributes. The square root of the total is found and added to the corresponding weighted value to give us the final euclidean distance. Attached in the Appendix, includes Figure M3 of the code snippet used to calculate the euclidean distance.

$$d(p_i, p_j) = \sqrt{\sum_{k=1}^n (p_{ik} - q_{jk})^2} + P_{pos\ i, pos\ j}$$

	Sergio Novak	Jorge Fernandes	Felix Smith	name
0	8.065597	10.916923	10.441917	Erik Almeida
1	7.950532	10.638472	10.679613	Nico Garcia
2	7.148469	9.558220	9.656693	Erik Smith

We utilised this Euclidean matrix to neatly generate a list of players which were closest to a particular player, using the function below:

```
def mostsimn(self, row, num, player_name):
    numeric_row = pd.to_numeric(row, errors='coerce').astype(float)

    # Filter out 0 values
    numeric_row = numeric_row.drop(player_name, errors='ignore')
    # Get the smallest values and their column names
    smallest = numeric_row.nsmallest(num)

    # Format the result as a list of "Column: Value" strings
    return [f"{col}" for col, val in smallest.items()]
```

This function generates the most similar players based on the smallest Euclidean distances. For each player, the function returns their closest peers in ascending order of Euclidean distance. Since the Euclidean distance between a player and themselves is 0, the player is removed from their own list.

Results

In order to visualize the euclidean matrix, we decided to use a heat map (Figure R1). Cells with lower values meant that the player on the row and the player on the column were more similar to each other. On the diagonal, all distances are 0, confirming that the euclidean distance between an entity and itself will always be 0.

For two player comparisons, we chose to use a radar chart. This is because we are comparing multiple features at a time which all started from a general 0 point. This can be seen in Figure R2 in the Appendix. Farther points mean higher scores for the specific features. The nearer the points of Player 1 was to Player 2, the more similar their scores were in that category. This allowed a user to see where exactly two players differed and where they were similar. Players that had smaller Euclidean distances between them had nearer points as compared to ones with farther distances.

To find the most similar players to a particular player, we picked the lowest n euclidean scores for that particular row, apart from 0, where n is a number between 1 and 10.

This is because a 0 value meant the player was being compared with themselves. The output can be seen in Figure R3 in the Appendix, ordered in ascending order of euclidean distance.

Discussion

The result generated aligned with our initial expectations when we began building the system. Our custom positional weight matrix ensured that players were grouped according to their similarity in position. For instance, finding the most similar players of a striker ensures the top ranked players have the same position, striker, and not a goal keeper or a defender. Nonetheless, it is entirely possible for a player in a different position to outrank a player in the same position, based on how close their attributes were.

Also, it was observed that when the number of players in the dataset increased, it was difficult to visualise the Euclidean distance using the heatmap. The boxes became less clear and harder to read, making it challenging to differentiate the various shades of colors across the boxes. We also noticed that if the overall weight is too high, eg. greater than 3, the system fails to account for the playing style of the player and blindly groups them according to their position tag.

Given more time, we would find the areas of the radar chart and compare them with the euclidean distance, to see which would be a better fit overall. Also a check would be made to see whether the various area values give an accurate unbiased description without having to be scaled accordingly.

Overall, this player similarity system allows us to acknowledge and appreciate how linear algebra could be used to develop a system that will help a team scout the next Lionel Messi or Cristiano Ronaldo.

Appendix

Methods

Figure M1

Row vectors

```

27
28         self.matrix_df = self.df[
29             [
30                 "pace",
31                 "dribbling",
32                 "passing_accuracy",
33                 "shooting",
34                 "tackling",
35                 "aerial_duels_won_pct",
36                 "positioning",
37                 "stamina",
38                 "goals_per90",
39                 "assists_per90",
40             ]
41         ]

```

Figure M2

Normalization

```

49     def euclidean_distance(self):
50         # scaling the values to ensure that each value is relevant when calculating the euclidian distance
51         # Using standardization:
52         scaler = StandardScaler()
53
54         self.matrix = scaler.fit_transform(self.matrix_df.to_numpy())
55

```

Figure M3

Euclidean distance

```

for i in range(len(self.matrix)):
    for j in range(len(self.matrix)):
        # difference between a given row and the remaining rows
        difference = self.matrix[i] - self.matrix[j]

        total = 0

        # calculate the total of square of the difference value
        # basically accomplishin (x2 - x1)**2 + (y2 - y1)**2
        for value in difference:
            total += pow(value, 2)
        euclidean_distance = math.sqrt(total)

        # get the encoded version of the position of both players.
        player1_position = self.df.loc[i, "position_encoded"]
        player2_position = self.df.loc[j, "position_encoded"]

        # obtain the penalty from the position distance matrix.
        position_penalty = position_distance[player1_position][player2_position] # type: ignore

        self.euclidean_matrix[i][j] = (
            euclidean_distance + position_weight * position_penalty
        ) # add the position penalty with a weight to the final euclidian distance.

```

Results

Figure R1

Heatmap

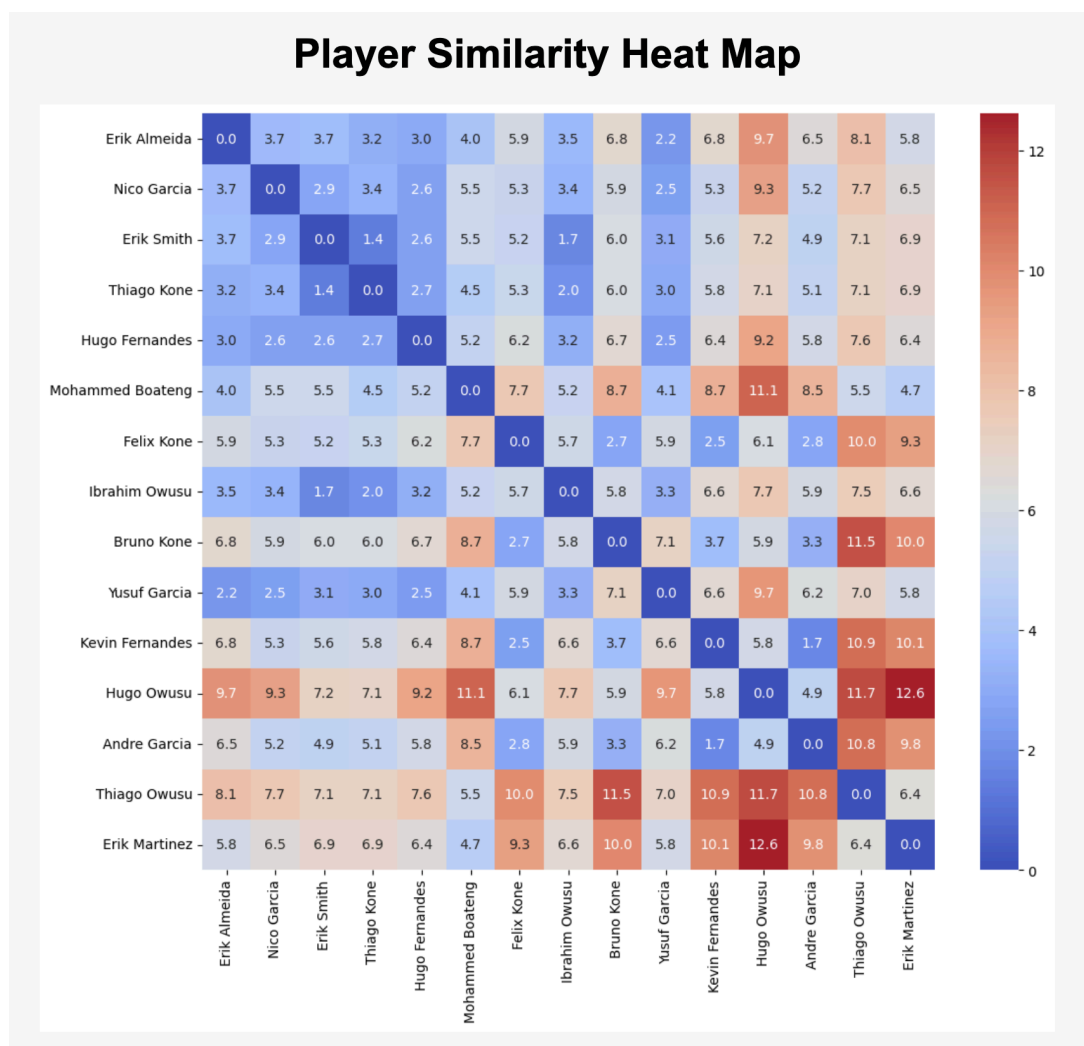
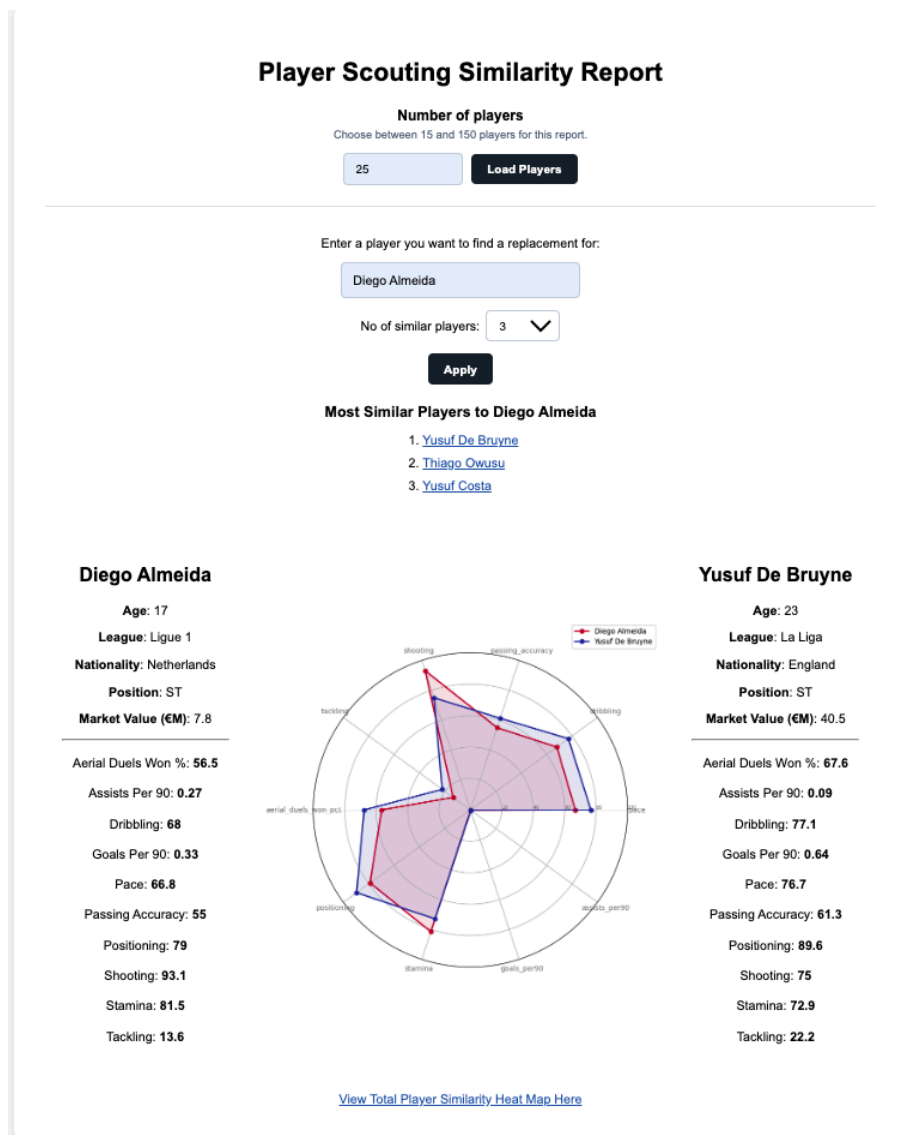


Figure R2

Radar charts



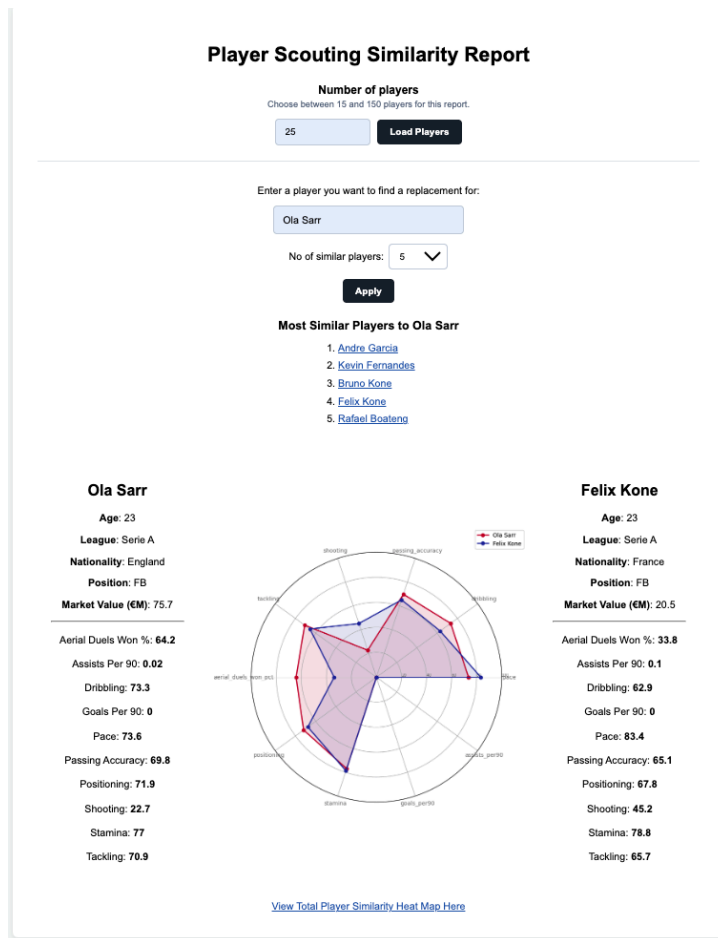


Figure R3

*Most Similar players***Player Scouting Similarity Report**

Enter a player you want to find a replacement for:

Erik Smith

No of similar players: 3 **▼****Apply****Most Similar Players to Erik Smith**

1. [Thiago Kone](#)
2. [Ibrahim Owusu](#)
3. [Hugo Fernandes](#)

[View Total Player Similarity Heat Map Here](#)**Player Scouting Similarity Report**

Enter a player you want to find a replacement for:

Hugo Fernandes

No of similar players: 3 **▼****Apply****Most Similar Players to Hugo Fernandes**

1. [Yusuf Garcia](#)
2. [Nico Garcia](#)
3. [Erik Smith](#)

[View Total Player Similarity Heat Map Here](#)

References

- Chugani, V. (2024, September 13). *Understanding euclidean distance: From theory to Practice*. Datacamp. <https://www.datacamp.com/tutorial/euclidean-distance>
- Highsoft. (n.d.). *Radar chart explained: When they work, when they fail, and how to use them right*. Highcharts. <https://www.highcharts.com/blog/tutorials/radar-chart-explained-when-they-work-when-they-fail-and-how-to-use-them-right/>
- Shaibu, S. (2024, October 15). *Normalization vs. standardization: Key differences explained*. DataCamp. <https://www.datacamp.com/tutorial/normalization-vs-standardization>
- Singh, S. (2018, March 5). *5 best substitutes in world football at the moment*. The Medium. <https://www.sportskeeda.com/football/5-best-substitutes-world-football-moment>
- Unknown. (n.d.). *Euclidean distance*. Euclidean Distance - distancia 0.0.1 documentation. <https://distancia.readthedocs.io/en/latest/Euclidean.html>